

RMD-Q Whitepaper

— One Assumption for Post-Quantum Keys and Signatures on Ultra-Constrained Devices

nul0 (Future Proofs Tech)
futureproofs@proton.me

September 2026

v0.1 — vision and design (companion: yellowpaper with the formal specification)
© 2026 nul0 (Future Proofs Tech). Licensed under GPLv3 (see LICENSE).

Abstract

Post-quantum standards secure the general case. Ultra-constrained devices — sensors, meters, wearables, field nodes — need cryptography that fits in kilobytes, sips energy, and still resists quantum attackers. RMD-Q is a research program toward that goal: a *new* hardness assumption coupling a lattice problem with a multivariate-quadratic problem over one secret, yielding *both* key exchange and signatures from *one* shared ≈ 3.9 KB codebase. This whitepaper explains the vision, the design, what the measurements say, and — just as carefully — what is not yet proven. The formal specification lives in the companion yellowpaper.

Contents

1	Why: the gap after standardization	2
2	The idea in one page	2
3	What the measurements say	2
4	What is not claimed	2
5	Roadmap	2
6	Call to action	3

1 Why: the gap after standardization

NIST’s post-quantum standards (ML-KEM, ML-DSA, SLH-DSA) are the right answer for servers, phones, and browsers. But a battery-powered sensor cannot comfortably carry two separate post-quantum stacks (one for key exchange, one for signatures), each with its own math, code, and audit surface. And the community’s security eggs sit in few baskets (module lattices plus hash-based backups) – assumption diversity, which NIST itself pursues in its additional-signatures process, is valuable.

RMD-Q answers both concerns at once: **one new assumption, two primitives, one codebase.**

2 The idea in one page

A secret s is sparse and short. Two public structures bind it:

1. a noisy linear relation $b = As + e$ (a lattice problem: recover s from noisy equations), and
2. a quadratic system $P(s) = 0$ (a multivariate problem: find a root).

A lattice-only attacker finds candidates that fail the quadratic check; an MQ-only attacker finds roots that violate the noisy equations. Breaking the scheme needs *both* solvers *combined* – that jointness is the assumption. From it we derive key encapsulation (Fujisaki-Okamoto transform, the standard recipe also behind ML-KEM) and signatures (Fiat-Shamir with a dual opening that proves both relations at once).

Key generation plants the quadratic system around the sampled secret, so it always terminates (rejection sampling would never finish at scale) – with measured statistical uniformity and a documented leakage analysis.

3 What the measurements say

On commodity hardware: full key exchange in ≈ 0.25 ms; public keys ≈ 0.8 KB, ciphertexts down to 800 B compressed. Decrypt-failure analysis gives rates below 2^{-235} where 2^{-138} is the bar, backed by 200 000 failure-free trials. The estimator models sit a few bits under ML-KEM at equal dimensions (stated openly, with tolerances). Every attack tried at small scale either broke the toy (proving the methods bite) or hit a documented wall – including a 114 GB out-of-memory kill on a 16-core box. Full tables: [docs/results/](#).

4 What is not claimed

No NIST security category. No deployment recommendation. No proof sketches yet (they are explicitly queued). The signature scheme is toy-grade. Side channels beyond measured timing shape are unstudied. Anyone telling you otherwise has not read Section 6 of the yellowpaper, which lists every non-claim in one place.

5 Roadmap

Short term: signature production parameters and proofs; compressed wire formats in C; embedded deployment diet (caller-provided buffers). Medium term: joint-attack analysis with stronger engines; external review; fpvll full-attack cross-checks. Long term: the assumption earns trust the only way assumptions ever do – by surviving years of public attack. We publish everything needed to attempt it.

6 Call to action

Break it. The challenge instances, reference code, attack scripts, and the exact list of open problems are in the repository. If the assumption falls, we want to know first – and the kill-phase log shows we report breaks, including our own design bugs, as prominently as wins.